

CSE327

Software Engineering

AKM iqtidar Newaz [IqN]

Lecture - 12

Factory Design Pattern

- Suppose, We have a Battleship superclass that extends into different types of subclasses like Destroyer, Carrier... etc.

```
#include <iostream>
using namespace std;
class Battleship
{
public:
    Battleship()
    {
        std::cout << "Battleship Created" <<
std::endl;
    }
    virtual void Fire() = 0;
    virtual void Steer() = 0;
};

class Destroyer : public Battleship
{
public:
    Destroyer()
    {
        std::cout << "Destroyer Created" << std::endl;
    }
    void Fire() override
    {
        std::cout << "Destoryer Fire" << std::endl;
    }
    void Steer() override
    {
        std::cout << "Destroyer Steer" << std::endl;
    }
};
```

main.cpp

```
class Carrier : public Battleship
{
public:
    Carrier()
    {
        std::cout << "Carrier Created" << std::endl;
    }
    void Fire() override
    {
        std::cout << "Carrier Fire" << std::endl;
    }
    void Steer() override
    {
        std::cout << "Carrier Steer" << std::endl;
    }
};

int main() {
    Destroyer d;
    Carrier c;
    d.Fire();
    c.Fire();
    return 0;
}
```

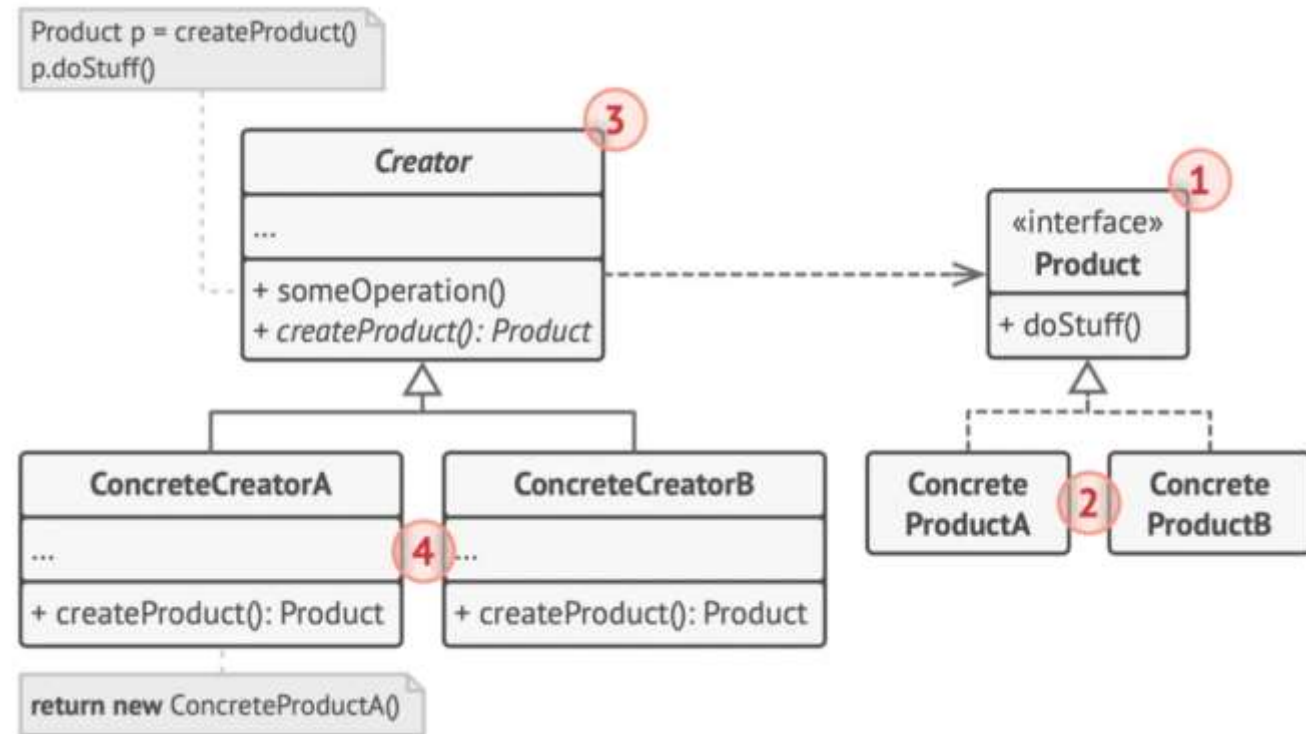
main.cpp

Factory Design Pattern

- Factory Method is a **creational design pattern** that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.
- Create objects without specifying the exact class of the object that will be created.
 - ❖ This means that before running the factory method, we don't know which type of object we are going to get after the method completes.
 - ❖ The type of the object is determined at runtime: This flexibility is crucial because if we knew the exact type at compile time, we would not need the factory method.
 - ❖ We need pointer or reference to a base class: We require a virtual factory method in the base class in order to call the correct instance methods of the object created by the factory method. This is polymorphism.
- It provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created.
 - ❖ This means, an interface declares a method for creating objects, but the actual implementation of that method is left to the subclasses. This allows subclasses to decide the type of object it wants to create without modifying the client code.

Factory Design Pattern

- Product means the superclass, or interface, the objects we ought to create implements.
- Concrete Products are the actual objects we ought to create.
- In our previous example, *BattleShip* class is the Product, and *Destroyer* and *Carrier* are the Concrete Products.
- Creator is the “**Factory Class**” which extends into **Concrete Creators** which will create the **Concrete Products** for us.
- There will be a **Concrete Creator** — *DestroyerCreator* that creates Destroyer’s, and similarly a *CarrierCreator* that creates Carrier’s



Factory Design Pattern Steps to Follow

- **Step 1:** Make all products follow the same interface. This interface should declare methods that make sense in every product.
- **Step 2:** Add an empty factory method inside the creator class. The return type of the method should match the common product interface.
- **Step 3:** In the creator's code find all references to product constructors. One by one, replace them with calls to the factory method, while extracting the product creation code into the factory method.
- **Step 4:** Now, create a set of creator subclasses for each type of product listed in the factory method. Override the factory method in the subclasses and extract the appropriate bits of construction code from the base method.

Example 1 - Code

```
#include <iostream>
using namespace std;

class Battleship
{
public:
    Battleship()
    {
        cout << "Battleship Created" << endl;
    }

    virtual void Fire() = 0;
    virtual void Steer() = 0;

    virtual ~Battleship()
    {
        cout << "Battleship destroyed" << endl;
    }
};
```

main.cpp

```
class Destroyer : public Battleship
{
public:
    Destroyer()
    {
        cout << "Destroyer Created" << endl;
    }
    void Fire() override
    {
        cout << "Destroyer Fire" << endl;
    }

    void Steer() override
    {
        cout << "Destroyer Steer" << endl;
    }
};
```

main.cpp

Example 1 - Code

```
class Carrier : public Battleship
{
public:
    Carrier()
    {
        cout << "Carrier Created" << endl;
    }
    void Fire() override
    {
        cout << "Carrier Fire" << endl;
    }

    void Steer() override
    {
        cout << "Carrier Steer" << endl;
    }
};
```

main.cpp

```
// run-time polymorphism
class ShipCreator
{
public:
    virtual Battleship *FactoryMethod() = 0;
    virtual ~ShipCreator() {}

    Battleship *CreateShip()
    {
        Battleship *ptr = this->FactoryMethod();
        return ptr;
    }
};

class CarrierCreator : public ShipCreator
{
public:
    Battleship *FactoryMethod() override
    {
        return new Carrier();
    }
};
```

main.cpp

Example 1 - Code

```
class DestroyerCreator : public ShipCreator
{
    main.cpp
public:
    Battleship *FactoryMethod() override
    {
        return new Destroyer();
    }
};
```

Output:

Battleship Created
Carrier Created
Carrier Fire
Carrier Steer

Second Battleship:
Battleship Created
Destroyer Created
Destroyer Fire
Destroyer Steer
Battleship destroyed
Battleship destroyed

```
int main()
{
    ShipCreator *creator = new CarrierCreator();
    Battleship *battleship1 = creator->CreateShip();
    battleship1->Fire();
    battleship1->Steer();
    main.cpp

    cout << "\nSecond Battleship:\n";

    creator = new DestroyerCreator();
    Battleship *battleship2 = creator->CreateShip();
    battleship2->Fire();
    battleship2->Steer();

    delete battleship1;
    delete battleship2;
    delete creator;
}
```

Example 2 (Restaurant Order Processing)

- Let's say we want to work on a restaurant ordering system, which allows customers to order two types of foods: Pizza and Pasta.
- A FoodFactory could be used to create different types of food items (e.g., PizzaFactory, PastaFactory). Each factory would handle the specifics of creating a particular food item, including ingredients, preparation steps, and pricing calculations.

Example 2 (Restaurant Order Processing)

```
#pragma once
#include <iostream>
#include <unordered_map>
using namespace std;
// Enum for food types
enumclass FoodType
{
    PIZZA,
    PASTA
};
// Abstract Product
class FoodItem
{
public:
    virtual void prepare() const = 0;
    virtual ~FoodItem() = default;
};
// Concrete Products
class CheesePizza : public FoodItem
{
public:
    void prepare() const override
    {
        cout << "Preparing Cheese Pizza" <<
endl;
    }
};
```

foodItem.h

```
class Pepperoni : public FoodItem
{
public:
    void prepare() const override
    {
        cout << "Preparing Pepperoni Pizza" <<
endl;
    }
};
class Spaghetti : public FoodItem
{
public:
    void prepare() const override
    {
        cout << "Preparing Spaghetti" << endl;
    }
};
```

foodItem.h

Example 2 (Restaurant Order Processing)

```
#pragma once
#include "foodItem.h"
// Abstract Factory

class FoodFactory
{
public:
    virtual FoodItem *createFood() const = 0;
};
// Concrete Factory
class CheesePizzaFactory : public FoodFactory
{
public:
    FoodItem *createFood() const override
    {
        return new CheesePizza();
    }
};
class PepperoniPizzaFactory : public FoodFactory
{
public:
    FoodItem *createFood() const override
    {
        return new Pepperoni();
    }
};
class SpaghettiPizzaFactory : public FoodFactory
{
public:
    FoodItem *createFood() const override
    {
        return new Spaghetti();
    }
};
```

foodFactory.h

```
#pragma once
#include "foodFactory.h"
#include "foodItem.h"
#include <unordered_map>

// Singleton Factory Registry
class FactoryRegistry
{
public:
    static FactoryRegistry &getInstance()
    {
        static FactoryRegistry instance;
        return instance;
    }
    void registerFactory(FoodType type, FoodFactory *factory) // Fixed function name
    {
        factories[type] = factory;
    }
    FoodFactory *getFactory(FoodType type) const
    {
        auto it = factories.find(type);
        if (it != factories.end())
        {
            return it->second;
        }
        return nullptr;
    }
private:
    FactoryRegistry() = default;
    unordered_map<FoodType, FoodFactory *> factories;
};
```

factoryRegister.h

Example 2 (Restaurant Order Processing)

```
#include "factoryRegister.h"
#include "foodFactory.h"
#include "foodItem.h"

                                main.cpp

int main()
{
    // Get the singleton instance of the FactoryRegistry
    FactoryRegistry &registry = FactoryRegistry::getInstance();

    // Register Factories
    registry.registerFactory(FoodType::PIZZA, new CheesePizzaFactory());
    registry.registerFactory(FoodType::PASTA, new SpaghettiPizzaFactory());

    // Simulate a Customer Order
    FoodType customerOrder = FoodType::PIZZA;

    FoodFactory *factory = registry.getFactory(customerOrder);

    if (factory)
    {
        FoodItem *food = factory->createFood();
        food->prepare();
    }
    else
    {
        cout << "No factory found for the requested food type!" << endl;
    }
    return 0;
}
```

Output:
Preparing Cheese Pizza

Example 3 - PizzaOrdering (Code)

```
#pragma once
#include <iostream>
#include <string>
#include <vector>
using namespace std;
```

pizza.h

```
class Pizza
{
public:
    Pizza() = default;
    virtual ~Pizza() = default;
    virtual void prepare() const;
    virtual void bake() const
    {
        cout << "Bake for 25 minutes at 350" << endl;
    }
    virtual void cut() const
    {
        cout << "Cutting the pizza into diagonal slices" << endl;
    }
    virtual void box() const
    {
        cout << "Place pizza in official PizzaStore box" << endl;
    }
protected:
    string name{};
    string dough{};
    string sauce{};
    vector<string> toppings{};
};
```

pizza.h

```
inline void Pizza :: prepare() const
{
    cout << "Preparing " + name << endl;
    cout << "\nTossing dough...\n";
    cout << "Adding sauce...\n";
    cout << "Adding toppings...\n";

    for (const auto & topping : toppings)
    {
        cout << " " + topping << endl;
    }
}
```

Example 3 - Pizza Store Ordering (Code)

```
#pragma once
#include "pizza.h"

class CheesePizza : public Pizza
{
public:
    CheesePizza();

    inline CheesePizza ::CheesePizza()
    {
        name = "Cheese Pizza";
        dough = "Thin Crust Dough";
        sauce = "Mariana Sauce";
        toppings.push_back("Grated Regiano Cheese");
        toppings.push_back("Shredded Cheese");
    }
}
```

cheesePizza.h

```
#pragma once
#include "pizza.h"

class ClamsPizza : public Pizza
{
public:
    ClamsPizza();

    inline ClamsPizza ::ClamsPizza()
    {
        name = "Clams Pizza";
        dough = "Thin Crust Dough";
        sauce = "Mariana Sauce";
        toppings.push_back("Grated Regiano Clam");
        toppings.push_back("Fresh Clams");
    }
}
```

```
#pragma once
#include "pizza.h"

class VeggiePizza : public Pizza
{
public:
    VeggiePizza();

    inline VeggiePizza ::VeggiePizza()
    {
        name = "Veggie Pizza";
        dough = "Thin Crust Dough";
        sauce = "Marinara sauce";
        toppings.push_back("Grated Regiano Veggie");
        toppings.push_back("Fresh veggies");
    }
}
```

clamsPizza.h

veggiePizza.h

Example 3 - Pizza Store Ordering (Code)

```
#pragma once
#include "cheesePizza.h"
#include "clamsPizza.h"
#include "pizza.h"
#include "veggiePizza.h"
#include <memory>
#include <string>
using namespace std;
```

```
class PizzaFactory
{
public:
    virtual Pizza *createPizza(string_view) const = 0; // Pure virtual function
    virtual ~PizzaFactory() = default;                // Virtual destructor
};
```

```
class VeggiePizzaFactory : public PizzaFactory
{
public:
    Pizza *createPizza(string_view name) const override
    {
        return new VeggiePizza();
    }
};
```

pizzaFactory.h

```
class CheesePizzaFactory : public PizzaFactory
{
public:
    Pizza *createPizza(string_view name) const override
    {
        return new CheesePizza();
    }
};
```

```
class ClamsPizzaFactory : public PizzaFactory
{
public:
    Pizza *createPizza(string_view name) const override
    {
        return new ClamsPizza();
    }
};
```

Example 3 - Pizza Store Ordering (Code)

```
#pragma once
#include "pizza.h"
#include "pizzaFactory.h"
#include <unordered_map>

class PizzaFactoryRegistry
{
public:
    static PizzaFactoryRegistry &getInstance()
    {
        static PizzaFactoryRegistry instance;
        return instance;
    }
    void registerFactory(const string &name, PizzaFactory *factory)
    {
        factories[name] = factory;
    }
    PizzaFactory *getFactory(const string &name)
    {
        auto it = factories.find(name);
        if (it != factories.end())
        {
            return it->second;
        }
        return nullptr;
    }
private:
    PizzaFactoryRegistry() = default;
    unordered_map<string, PizzaFactory *> factories;
};
```

pizzaFactoryRegistry.h

Example 3 - Pizza Store Ordering (Code)

```
#pragma once
#include "pizzaFactoryRegistry.h"
#include "pizzaFactory.h"
#include "cheesePizza.h"
#include "clamsPizza.h"
#include "veggiePizza.h"

class PizzaStore
{
public:
    PizzaStore(PizzaFactory *f) : factory(f)
    {
        // Get singleton instance of Factory Registry
        PizzaFactoryRegistry &registry = PizzaFactoryRegistry::getInstance();

        // Register Concrete Factories
        registry.registerFactory("CheesePizza", new CheesePizzaFactory());
        registry.registerFactory("ClamsPizza", new ClamsPizzaFactory());
        registry.registerFactory("VeggiePizza", new VeggiePizzaFactory());
    }
    Pizza *orderPizza(string_view type);
private:
    PizzaFactory *factory = nullptr;
};
inline Pizza *PizzaStore::orderPizza(string_view type)
{
    auto pizza = factory->createPizza(type);
    pizza->prepare();
    pizza->bake();
    pizza->cut();
    pizza->box();
    return pizza;
}
```

pizzaStore.h

Example 3 - Pizza Store Ordering (Code)

```
#include "pizzaFactory.h"
#include "pizzaStore.h"
#include "veggiePizza.h"
#include "clamsPizza.h"
#include "pizzaFactoryRegistry.h"
#include "cheesePizza.h"
#include <memory>
int main()
{
    auto factory = new CheesePizzaFactory(); // Create Facotry
    auto pizzaStore = PizzaStore(factory); // Create store and pass it factory
    string customerOrder1 = "CheesePizza";
    auto pizza = pizzaStore.orderPizza(customerOrder1); // Order pizza from store
    cout << "\n\t";
    auto factory2 = new ClamsPizzaFactory();
    auto store = PizzaStore(factory2);
    string customerOrder2 = "ClamsPizza";
    auto pizza2 = store.orderPizza(customerOrder2);
}
```

main.cpp

Output:

Preparing Cheese Pizza

Tossing dough...

Adding sauce...

Adding toppings...

Grated Regiano Cheese

Shredded Cheese

Bake for 25 minutes at 350

Cutting the pizza into diagonal slices

Place pizza in official PizzaStore box

Preparing Clams Pizza

Tossing dough...

Adding sauce...

Adding toppings...

Grated Regiano Clam

Fresh Clams

Bake for 25 minutes at 350

Cutting the pizza into diagonal slices

Place pizza in official PizzaStore box